

Section 1: Database Architecture

Section 1: Database Architecture & Connection Layer

This layer dictates how data is stored, how financial integrity is maintained, and how the system boots up.

1. `schema.sql` (The Master Architecture)

This file defines the PostgreSQL tables. The logic here is heavily focused on financial immutability and physical hardware mapping.

- **users Table:**
 - *Logic:* Stores basic auth data. Uses `bcrypt` for password hashing.
 - *Key Feature:* The `role` column (`manager` vs `staff`). This single column dictates what UI elements render in Flutter and which Express routes allow access.
- **medicines_inventory Table:**
 - *Logic:* The core stock ledger.
 - *Key Features:* Contains `unit_cost` (what the pharmacy pays) and `pay_rate` (what the pharmacy charges). Crucially, it includes `storage_location` (e.g., "A-01-B")—this is the exact string the Hardware team uses to map the physical bin coordinates for the robotic arm.
- **invoices_fawateer & invoice_items Tables (Purchasing):**
 - *Logic:* Tracks stock coming *in* from suppliers.
 - *The "Why":* The system supports draft invoices. If an invoice is marked `PENDING`, the items are saved, but the `medicines_inventory` stock quantities do *not* increase. The trigger to physically increase stock and update the `unit_cost` only fires when the invoice status is flipped to `PAID`.
- **prescriptions & prescription_items Tables (Sales):**
 - *Logic:* Tracks stock going *out* to patients.
 - *The "Why":* Look at `pay_rate_at_sale` and `unit_cost_at_sale`. When a prescription is checked out, the system copies the *current* prices from the inventory table and locks them into this sales row. If the price of Panadol goes up next month, your historical profit margins for today remain mathematically accurate.
- **robot_jobs Table:**
 - *Logic:* The bridge between software and hardware. It links to a `prescription_id` and stores a `pick_sequence` as a JSONB array. It tracks the hardware state (`PENDING`, `DISPATCHED`, `COMPLETED`, `ABORTED`).

2. `db.ts` (The Connection Pool)

- *Logic:* Initializes a `pg` (node-postgres) Connection Pool.
- *The "Why":* Instead of opening and closing a new database connection for every single HTTP request (which would crash the server under heavy load), the Pool

keeps a set of connections open and reuses them. It exports a `query` helper function used across all Express routes.

3. `db-init.ts` (The Bootstrapper)

- *Logic*: A standalone script that reads `schema.sql`, splits it by semicolons, and executes it sequentially.
- *The "Why"*: This makes the project portable. The examiners or anyone on your team can clone the repo, run this script, and instantly have a fully structured database populated with the default manager/staff accounts without needing to manually configure pgAdmin.

Section 2: Node.js Gateway

Section 2: Node.js Gateway & Security Layer

This layer acts as the absolute authority for the entire system. Neither the frontend apps, the AI engine, nor the hardware can perform any action without passing through this gateway and proving their identity via cryptographic tokens.

1. `index.ts` (The Central Nervous System)

This is the entry point for the backend server.

- **Express Setup:** It initializes `express` with `cors` and `json` middleware to allow cross-origin requests from the Flutter apps.
- **Router Mounting:** It mounts all modular sub-routers (`/api/auth`, `/api/robot`, etc.). This **deoupling** allows different team members to work on separate backend files without breaking the main server.
- **WebSocket Integration:** It initializes a `WebSocketServer` (`wss`) alongside the HTTP server.
 - **The Logic:** It exports `broadcastEvent`, a helper function that pushes data to all connected clients.
 - **The "Why":** Unlike REST (where the app must ask for updates), WebSockets allow the server to "push" robotic arm status updates to the dashboard instantly.

2. `routes/auth.ts` (Authentication & Hashing)

This file handles user login, registration, and session generation.

- **Login Logic:**
 - It uses `bcrypt` to compare the incoming password with the stored hash.
 - **Dummy Hash Migration:** It contains a specific block that detects if a user is still using the hardcoded "dummy" hash. If found, it automatically hashes the password correctly, updates the database, and forces a re-login.
 - **The "Why":** This is a critical security upgrade path for moving from a prototype (with dummy data) to a production-ready system.
- **JWT Generation:** Upon success, it generates a **JSON Web Token (JWT)** containing the `userId` and `role`. This token is signed with a `JWT_SECRET` and expires in 8 hours.

3. `middleware/auth.ts` (The Bouncer)

This middleware intercepts every incoming request to protected routes.

- **Function: `authenticateToken`:** It extracts the token from the `Authorization: Bearer <token>` header.
- **Verification:** It uses `jwt.verify` to check the token's validity. If the token is expired or tampered with, it returns a `401 Unauthorized` or `403 Forbidden` before the request ever touches your database.

4. `services/token_store.dart` (Frontend Secure Storage)

This is the Flutter-side logic for managing the tokens sent by the Node server.

- **Hardware Encryption:** It uses `FlutterSecureStorage`.

- **Logic:** On Android, it uses **AES** encryption within **EncryptedSharedPreferences**. On iOS, it uses the **Keychain**.
- **The "Why":** Standard app storage is plain text and easy to hack. Secure storage uses the device's specialized security hardware (Keystore/Enclave) to ensure the JWT cannot be stolen by other apps on the device.
- **Device ID Generation:** It includes a **getOrCreateDeviceId** function that generates a unique UUID v4 for the tablet. This allows the backend to track exactly which physical device is interacting with the robot.

5. **services/auth_service.dart** (Session Management)

This bridges the gap between the raw token and the UI.

- **Role Mapping:** It maps the backend string (**manager** or **staff**) to a Dart **UserRole** enum.
- **IsManager Helper:** Provides a global **AuthService.isManager** getter used by **app_shell.dart** to hide or show sensitive tabs like "AI Command Center" and "Operational Reports".

6. **api_service.dart** (The REST Client)

A bulletproof HTTP wrapper used for all communication.

- **_headers Function:** Automatically fetches the JWT from **TokenStore** and attaches it to every single request as a Bearer token.
- **Error Handling:** It maps raw HTTP status codes to custom exceptions like **NetworkException** or **ApiException**.
- **The Gateway Host:** It allows the user to dynamically change the **gatewayHost** (the IP of the backend PC) without re-coding the app.

Section 3: Inventory, Invoicing

Section 3: Inventory, Invoicing & The Financial Logic

This section governs the "Money Out" side of the pharmacy and is designed to ensure that the digital inventory exactly matches the physical shelves.

1. Invoices & The Stock Update Trigger ([express_routes/invoices.ts](#))

This file manages the purchasing of stock from suppliers.

- **The Logic (The "PAID" Trigger):** When an invoice is created, it is often in a `PENDING` state. The critical logic here is that physical stock quantities in the `medicines_inventory` table do **not** increase when a pending invoice is saved.
- **Inventory Integration:** The inventory update *only* occurs when the invoice status is flipped to `PAID`.
- **The "Why" (Phantom Stock Prevention):** This prevents "phantom" inventory. If you scan an invoice for a delivery that hasn't arrived yet, you don't want the system to think you have those medicines. If the robot attempts to dispense a medicine that is digitally "in stock" but physically missing, the entire automated line would jam.
- **Unit Cost Updating:** When an invoice is paid, the system doesn't just add quantity; it overwrites the `unit_cost` of the medicine with the latest price from that specific invoice. This ensures your profit margins always reflect the most recent cost basis.

2. Inventory Health & HCI ([inventory_screen.dart](#))

This screen provides a real-time view of every pack in the pharmacy.

- **HCI (Human-Computer Interaction):** Instead of showing raw expiration dates, the app uses a `StockStatus` enum to calculate health badges: **Healthy**, **Warning**, **Critical**, or **Out of Stock**.
- **Visual Cognitive Load:** Pharmacists are busy; the use of color-coded progress bars for stock levels and badges for expiry labels (e.g., "15 days") allows a user to scan a list of 500 items and identify risks in milliseconds.
- **Ad-Hoc Hardware Control:** This screen includes a unique "Dispatch Robot" button for individual medicines.

3. The "Bulletproof" Excel Importer ([express_routes/medicines.ts](#))

This feature allows the pharmacy to migrate from old systems or spreadsheets instantly.

- **Logic (Keyword Mapping):** Pharmacists often use messy Excel files. The code uses a `findVal` helper that hunts for keywords like `['packs', 'كمية', 'qty']`.
- **The "Why" (Robustness):** It doesn't matter what the pharmacist named their columns; the code dynamically maps them to the PostgreSQL schema.
- **Data Deduplication:** If an item with the same `trade_name` already exists, the importer uses an `UPDATE` query to add the new quantity to the old quantity. If it's new, it generates a unique `EXC-` barcode and creates a new row.

4. Financial Reporting ([express_routes/reports.ts](#))

While the AI predicts the future, this file reports the absolute truth of the present.

- **Logic (Rolling 365-Day Metrics):** It aggregates data for `total_revenue`, `total_profit`, and `units_dispensed` specifically within the last year.

- **Logic (Waste Calculation):** It identifies medicines where `expiration_date < CURRENT_DATE` and sums their total value.
- **The "Why":** This provides the **financial baseline**. A pharmacy manager needs to know exactly how much money is sitting on the shelves (Capital Asset) versus how much is being lost to expiry (Waste).

Section 4: Prescriptions, Sales

Section 4: Prescriptions, Sales & The Two-Pass OCR Engine

This section explains how stock securely leaves the pharmacy and the advanced logic required to translate messy, real-world handwriting into clean database records. This is where you demonstrate the system's ability to handle **unstructured data**.

1. The Sales Lifecycle ([express_routes/prescriptions.ts](#))

This file governs the "Money In" operations and enforces strict inventory safety rules.

- **Public vs. Authenticated Routes:** The system has a public [POST /](#) route that allows the **Roshetty** app to submit prescriptions without a staff login. All other routes (deletion, linking, checkout) require a pharmacist's JWT.
- **The Price-Locking Snapshot:** When an item is added to a prescription, the system fetches the current [pay_rate](#) and [unit_cost](#) from inventory and saves them directly into the [prescription_items](#) row.
 - **The "Why":** This ensures **historical financial integrity**. If prices change next month, today's profit margin reports remain 100% accurate because the sale was "locked" at the time of entry.
- **Final Checkout Validation:** The [POST /:id/checkout](#) route performs a last-second check on [quantity_on_hand](#) and [expiration_date](#).
 - **The "Why":** This is a critical safety failsafe. It prevents the robotic arm from attempting to dispense medication that has expired or has run out of stock since the order was first scanned.

2. The Intelligence Engine: Two-Pass OCR ([ocr_core.py](#))

Developed by Demiana, this script is the bridge between a physical image and digital data.

- **Step 1: Aggressive Cleaning:** The [aggressive_clean](#) function strips out "medical noise" words like **tablet**, **daily**, **mg**, and **vial**. This leaves only the core chemical or brand names for matching.
- **Step 2: The Two-Pass Matching Engine:**
 1. **Pass 1 (Blazing Fast):** A standard fuzzy match using [SequenceMatcher](#) to find high-confidence hits.
 2. **Pass 2 (OCR Visual Fix):** If confidence is low, the engine uses a custom [OCR_VISUAL_MAP](#) to account for common character misreads (e.g., visual similarity between '1' and 'l', or '5' and 's').
- **The "Why":** OCR is probabilistic, not deterministic. By cleaning the noise and fixing visual character swaps, you significantly increase the accuracy of the medicine identification before it ever reaches the pharmacist.

3. Bridging the Gap: The Fuzzy Linker ([medicine_linker_sheet.dart](#))

Even the best AI can fail if handwriting is illegible.

- **The Logic:** If the OCR returns a medicine name that doesn't exist in your database (e.g., [medicine_id: 0](#)), the UI flags it with an amber warning. The pharmacist then opens the [MedicineLinkerSheet](#).
- **The Technical Flex:** The backend uses the PostgreSQL [similarity\(\)](#) function to perform a **Fuzzy Search**, returning the top 5 closest matches for the pharmacist to manually link.

- **The "Why":** This maintains **Data Integrity**. You never allow a "floating" text string to be dispensed. Every item must be tied to a real database ID to ensure the robot moves to the right bin and stock is deducted correctly.

4. Roshetty: Public Access & Security ([express_routes/public.ts](#))

The Roshetty app allows patients to interact with the pharmacy ecosystem safely.

- **The Public Endpoint:** Patients use [/api/public/check-prescription](#) to verify if their meds are available.
- **The "Why" (Security):** This route is **Rate Limited** (max 30 requests per minute) to prevent automated "scraping" of your inventory data. It only returns availability status and prices—it never exposes sensitive data like supplier names or unit costs.

Section 5: Predictive AI & Operational Analytics

Section 5: Predictive AI & Operational Analytics

This section explains how the system transitions from a static management tool to an **Intelligent Ecosystem** by using historical data to predict future needs.

1. `app.py` / `main.py` (The Intelligence Microservice)

These Python servers (Flask/FastAPI) act as the brain of the project, hosting the machine learning models.

- **Demand Forecasting (XGBoost):**
 - **Logic:** The system loads historical sales data from Excel files (`1.xlsx` to `12.xlsx`) into RAM. An **XGBoost Regressor** (`demand_xgb.json`) analyzes these monthly trends to predict the total units required for the upcoming month.
 - **The "Why":** Standard linear averages don't account for seasonality in medicine sales. XGBoost handles non-linear patterns, allowing for high-precision procurement.
- **Smart Reorder (Q-Learning / Reinforcement Learning):**
 - **Logic:** Uses a **Q-Table** (`q_table_reorder.joblib`) where the "state" is current stock and the "action" is the reorder quantity. If the model lacks data, it utilizes a statistical fallback: $(\text{Mean Sales} / 30 * 7 \text{ days}) + (1.5 * \text{Standard Deviation})$.
 - **The "Why":** This balances the cost of overstocking against the risk of life-saving medicine running out. Including standard deviation accounts for sales volatility, not just volume.
- **Integrity Audit (Autoencoder):**
 - **Logic:** A neural network (`anomaly_ae.keras`) trained to "reconstruct" shift data. If the reconstruction error (MSE) is too high, or if a cash difference exceeds 300 EGP, it flags the shift as an anomaly.
 - **The "Why":** This is a fraud-prevention and human-error detection tool that identifies unusual patterns standard accounting might miss.

2. `express_routes/ai.ts` (The Node.js Proxy)

This backend file acts as the secure bridge between the Flutter app and the Python AI microservice.

- **Logic:** It receives authenticated **GET** requests from the dashboard and uses `fetch` to proxy them instantly to the Python ports (5000/5001).
- **The "Why":** By isolating the AI models behind a Node.js proxy, you prevent **CORS** issues and ensure that sensitive predictive data is only accessible to users with a valid `Manager` JWT.

3. `ai_center_screen.dart` (Actionable Visualization)

The Flutter implementation that makes complex AI data useful for a human pharmacist.

- **Logic:** Uses the `fl_chart` library to render complex datasets like the **Monthly Demand Trend** (Line Chart) and **Category Health Matrix** (Pie Chart).
- **The HCI Flex (Clearance Promos):** Instead of just listing expiring items, the UI fetches clearance recommendations. It calculates a dynamic discount (20% to 50%) based on the financial loss risk predicted by the Python engine.

- **The "Why":** Information is only valuable if it is **actionable**. By suggesting specific clearance prices, the system helps the pharmacist liquidate risky stock and recover capital before it expires.

4. **operational_reports.dart** (The Baseline Truth)

While Ghada's AI predicts the future, this screen reports the absolute reality of the last 365 days.

- **Features:** It displays rigid metrics for **Total Revenue**, **Net Profit**, **Units Dispensed**, and **Waste Value**.
- **The Logic:** It pulls from **reports.ts**, which uses specific SQL **SUM** and **COUNT** queries over a rolling 1-year interval.
- **The "Why":** It provides the "Control Group" for the AI. A manager can compare the actual rolling revenue against the AI's demand forecast to verify the system's accuracy over time.

Section 6: The Hardware Integration Guide

Section 6: The Hardware Integration Guide

This final section is specifically for the **Hardware Team**. It explains how to replace the existing software "mock" simulation with actual microcontroller logic (ESP32/Raspberry Pi) to control the physical robotic arm.

1. Ripping Out the "Fake" Robot Simulation

Currently, `express_routes/robot.ts` uses JavaScript `setTimeout` blocks to "fake" the robot's movement so the UI can be tested.

- **Your Task:** Open `express_routes/robot.ts` and delete the inner `setTimeout` blocks in the `POST /dispatch`, `POST /retry/:jobId`, and `POST /dispatch-adhoc` routes.
- **The New Logic:** The backend should now only insert the job into the `robot_jobs` table and broadcast the `DISPATCHED` signal. It must **not** automatically mark the job as `COMPLETED`. The physical robot is now the only authority that can confirm the task is finished.

2. The Communication Contract (WebSocket)

The robot controller must maintain a persistent, low-latency connection to the Node.js server to receive instant commands.

- **WebSocket URL:** `ws://<SERVER_IP>:4000`.
- **The "Wake Up" Signal:** When a pharmacist taps "Dispatch" in Flutter, the server broadcasts a JSON event. Your hardware must listen for:
 - JSON
 - `{ "type": "ROBOT_ACK", "jobId": "JOB-12345", "status": "DISPATCHED", "message": "Robot has acknowledged job" }`
 - ````[cite: 27, 30]`.

3. Fetching the Physical Grid Coordinates (HTTP GET)

The WebSocket message only notifies you that a job exists. To get the physical coordinates, your hardware must perform a standard HTTP GET request.

- **Endpoint:** `GET http://<SERVER_IP>:4000/api/robot/jobs/<jobId>`.
- **The Data Payload:** You must parse the `pick_sequence` JSON array:
 - **bin:** The coordinate string (e.g., `"A-01-C"`). Your firmware must map this string to specific stepper motor steps on your X, Y, and Z axes.
 - **qty:** The number of packs to drop.

4. Reporting Physical Success (New API Route Requirement)

Once the items have physically dropped into the dispensing tray, the hardware **must** tell the database it is finished so the UI updates and inventory is deducted.

- **Action Required:** You must add a new route in `robot.ts` (e.g., `POST /api/robot/jobs/:jobId/complete`).
- **Mandatory Logic for the Route:**
 1. **Deduct Inventory:** Subtract the dispensed quantities from the `medicines_inventory` table.
 2. **Finalize Job:** Update the `robot_jobs` table status to `COMPLETED`.

3. **UI Refresh:** Call `broadcastEvent({ type: 'ROBOT_ACK', status: 'COMPLETED' })` so the iPad turns green and the activity log updates.

5. Handling Safety Interrupts (Emergency Abort)

If a user taps the red **"Stop Arm"** button on the dashboard, the server broadcasts an **ABORTED** status.

- **Hardware ISR:** Your WebSocket listener must treat `status: 'ABORTED'` as a high-priority interrupt. Your code should immediately cut power to the motor drivers or execute a "Safe Return to Home" sequence to prevent mechanical damage or injury.

Section 7: Navigation Architecture & Backend

Section 7: Navigation Architecture & Backend Auditing

This section explains how the system manages the user experience across different screen sizes and how the backend maintains an audit trail of every dollar earned during a shift.

1. Responsive Navigation (`app_shell.dart`)

This file acts as the primary layout wrapper for the entire APMS dashboard, ensuring it works on tablets, iPads, and desktop computers.

- **Logic (Role-Based Visibility):** The `visibleItems` getter filters the `navItemsList` dynamically. If the current user is not a **Manager**, the "Operational Reports" and "AI Command Center" tabs are stripped from the menu before the UI is built.
- **The "Why" (UX Security):** It prevents staff members from even seeing that sensitive financial or predictive data exists, reducing cognitive clutter and improving security.
- **Adaptive UI:** On desktop (width $\geq$ 960), it renders a fixed **Sidebar**. on mobile/tablet, it switches to a **NavigationBar** at the bottom.
- **The "Why" (HCI):** Pharmacists often carry iPads while moving between shelves. A bottom navigation bar allows for one-handed operation (thumb-reach) which is a core **Human-Computer Interaction** principle for mobile devices.

2. KPI Logic & Risks (`DashboardKpiRow.dart`)

This widget defines the "Pulse" of the pharmacy seen on the home screen.

- **Logic (The "Risk Detected" Trigger):** The `_ExpiringKpiCard` checks if the `expiringSoon` count is > 0 . If it is, the card's border and icon change to **Red** (`AC.redBg`), and the label flips from "ALL CLEAR" to "RISK DETECTED".
- **The "Why" (Preattentive Processing):** By using specific colors and high-contrast warnings, the UI exploits human "preattentive processing"—meaning the pharmacist's brain recognizes a problem before they even read the numbers.

3. Backend Shift Auditing (`routes/shifts.ts`)

While you removed the frontend reconciliation modal, the backend logic remains as a critical audit tool for managers.

- **The `reconcile` Function:** It calculates the `expectedAmount` by summing all `prescription_items` marked as `DISPENSED` within the last 12 hours.
- **The Difference Logic:** It compares the pharmacist's `actualAmount` (input) against the `expectedAmount` and saves the `difference` to the `shift_reconciliations` table.
- **The "Why" (Fraud Prevention):** This is a **Blind Reconciliation**. Because the backend doesn't tell the staff member what the "expected" number is, they cannot simply type that number to hide missing cash. This data then feeds Ghada's **Autoencoder** to detect if a specific staff member has a pattern of "missing" money.

4. Roshetty App Connectivity (`api_service_2.dart`)

This file is the specific bridge for the patient mobile app.

- **Function: `checkPrescription`:** It posts a list of medicine names to the gateway.
- **The Logic:** It sets a strict 8-second timeout (`_timeout`) and includes the `ngrok-skip-browser-warning` header.

- **The "Why":** Since the backend is hosted on a local PC using Ngrok for the demo, this header prevents the Roshetty app from crashing on the Ngrok "warning" page, ensuring a smooth connection for the examiners.

APMS & Roshetty (Summary)

APMS & Roshetty (Summary)

1. The Core Vision: Bridging Architecture and Empathy

Our project is not just a software tool; it is an **Intelligent Pharmacy Ecosystem**. We identified a gap where traditional pharmacy systems are either purely manual or technically rigid. Our mission was to bridge technical architecture with user empathy by creating a system that manages inventory securely, predicts future needs through AI, and dispenses medication physically via robotics.

2. The Integrated Workflow (The Life of an Order)

The system operates through a seamless loop across four distinct technical layers:

1. **Patient Entry (Roshetty)**: A patient remotely scans a prescription using our mobile app. The **OCR Engine** (Demiana's work) extracts the medicine names.
2. **Gateway Orchestration (Node.js)**: The names are sent to our **Central Gateway**. The backend uses **PostgreSQL Fuzzy Search** to link the raw text to actual inventory IDs.
3. **Intelligence Layer (Python AI)**: Simultaneously, **Ghada's Predictive Models** analyze this sale against 12 months of historical data. The system calculates if this sale triggers a **Smart Reorder** (Q-Learning) or impacts the **Demand Forecast** (XGBoost).
4. **Physical Execution (Robotic Arm)**: Once the pharmacist approves, the Node.js server broadcasts a **WebSocket signal** to the robotic arm. The arm moves to the specific **storage_location** (Bin), dispenses the items, and reports completion back to the system.

3. Three Defensive Pillars for the Examiners

When the panel asks why we made certain technical choices, here are the "Win" conditions:

- **Pillar 1: Data Integrity & Financial Security:**
 - We implemented **ACID Transactions** in our Node.js backend to ensure stock is never half-updated.
 - We used **Price-Locking** at the moment of checkout, so our historical profit reports are 100% immune to future price inflation.
- **Pillar 2: Defensive Human-Computer Interaction (HCI):**
 - Our dashboard uses **Preattentive Visual Cues** (Red/Amber/Green status badges) to reduce the cognitive load on busy pharmacists.
 - The patient app uses a **Studio Ghibli-inspired "Cozy" palette** to reduce medical anxiety—a key HCI design principle.
- **Pillar 3: Modular, Decentralized Scaling:**
 - By using a **Microservice Architecture** (Node.js for logic, Python for AI, Flutter for UI), we can scale any part of the system independently. If we upgrade the robot, the software doesn't break. If we change AI models, the dashboard remains stable.

4. Technical Conclusion

We have built a system that is **Secure** (JWT/Hardware Encryption), **Scalable** (REST/WebSockets), and **Intelligent** (Predictive Modeling). It transforms the pharmacy from a simple shop into a data-driven healthcare hub.

Notes

The Problem Statement: Bridging the gap between rigid management systems and human empathy.

The Tech Stack:

- **Frontend:** Flutter (Cross-platform, Adaptive UI).
- **Backend:** Node.js, Express, TypeScript, PostgreSQL.
- **AI:** Python, XGBoost, TensorFlow, Q-Learning.[cite: 41, 42]
- **Hardware:** Robotics (WebSockets, Arduino/ESP32).

Core Security: Highlight the use of **JWT** and **OS-native hardware encryption** for credential storage.

Financial Integrity: Explain the **Price-Locking** logic in `prescription_items` and **ACID Transactions** for stock updates.

Backend Port: Defaults to `4000`. [cite: 39]

AI Port: Defaults to `5000` (FastAPI) and `5001` (OCR).

Local Gateway IP: Ensure the `api_service.dart` is configured to allow LAN IP changes in the settings screen.